

FINAL INFORMATICA I			Diciembre 2022	
Nombre y Apellido	Nº Legajo	Calificación	Docente Evaluador	
			Nombre	Firma

Sección teórica

No se permite realizar con PC

Punto 1.-

En base a las siguientes declaraciones, indique cuales de las analogías presentes en la tabla son válidas al momento de acceder a una posición del Array, ya sea a la dirección (ubicación) o a su contenido (al dato en sí).

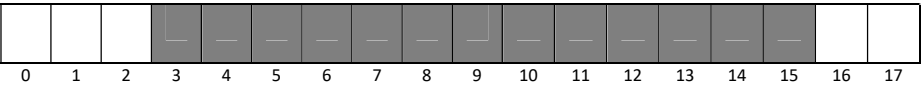
```
#define NN 20
short intvec[NN];
shortint *p=vec;
```

				V/F
A	p[10]	≡	*(&Arr[10])	
B	&p [10]	≡	*(p+10)	
C	&p [10]	≡	(p+8+2)	
D	*Arr[10]	≡	*(p+10)	
E	p+40	≡	Arr+40	
F	Arr[10]	≡	*Arr+10	
G	&Arr[10]	≡	(Arr+10)	
H	*(&Arr[10])	≡	*p+10	

Punto 2.-

Se posee una función para el procesamiento de Arrays unidimensional, compatible con el que deseamos procesar. Pero solo necesitamos procesar los elementos que van desde la posición 3 hasta 2 antes que finalice el array.

Por ejemplo, en el siguiente array de 18 elementos se estaría ordenando la zona gris:



Sabiendo que:

- la cantidad de elementos en nuestro array es **size** (en el ejemplo es 18).
- El array del cual solo se debe procesar una parte es **vec**.
- Que la función recibe primero la cantidad de elementos y luego la dirección del array

¿Cuáles de las siguientes invocaciones son válidas?

(puede haber 0, 1 o más de una respuesta correcta – marque todas las que considere correctas).

		V/F
A	func_pz (size-5, vec+5);	
B	func_pz (size-5, vec+=3);	
C	func_pz (size+5 ,&vec[3]);	
D	func_pz (size-5,&(*vec+3));	
E	func_pz (size-5, vec[3]);	
F	func_pz (size-5, vec+3);	

FINAL INFORMATICA I			Diciembre 2022	
Nombre y Apellido	Nº Legajo	Calificación	Docente Evaluador	
			Nombre	Firma

Sección Práctica

Como parte de un equipo de desarrollo, debe realizar una serie de funciones para manejar una cola de mensajes (datos) multi destino.

Como toda cola de mensaje (o de datos) el primer elemento insertado es el primero en ser retirado siempre y cuando cumpla con la condición del destinatario.

El modelo de memoria a utilizar para el manejo de la cola de mensajes queda a su criterio, pudiendo ser una lista simplemente enlazada, un bloque de memoria compacta u otro. Si es mandatorio que la misma se encuentre en el HEAP (memoria dinámica)

En ningún caso se permite el uso de variables globales.

Funciones a desarrollar: Las siguientes son las funciones mínimas a desarrollar. Si acorde al modelo de memoria seleccionado requiere de otras funciones, indíquelo y desarróllelas.

Función: `int init ( __otros_parámetros__ );`

La función o macro `init` debe implementar todo aquello que sea necesario para dejar la cola de mensajes lista para ser utilizada.

En el caso que su diseño no requiera de esta función, no la desarrolle e indique las condiciones que se deben cumplir para que la cola de mensajes este disponible para su uso.

Función: `int pushMulti ( __otros_parámetros__ , const data_t * pdat , uint32_t dest );`

La función `pushMulti` debe insertar el dato en la cola de mensajes, indicando mediante un campo de bits (*bitwise*) los destinos del mensaje.

- `pdat`: dato a insertar en la cola de mensaje.
- `dest`: valor compuesto por un bit wise que indica los destinos a donde está dirigido el mensaje.
- `__otros_parámetros__`: todo aquello que considere necesario para la operación de la función.

La función debe retornar 1 si el dato es insertado correctamente. Caso contrario debe retornar 0.

Ejemplo de uso:

```
#define DEST_01      0x0001
#define DEST_02      0x0002
.....
#define DEST_06      0x0020
.....
data_t dato;
.....
ret=pushMulti ( __otros parámetros__ , &dato , DEST_01|DEST_03|DEST_04|DEST_06 );
```

Función: `int popDest ( __otros_parámetros__ , data_t * pdat , uint32_t dest );`

La función `popDest` debe extraer el dato de la cola de mensajes correspondiente al destino indicado (`dest`).

- `pdat`: buffer en donde dejar el dato.
- `dest`: valor compuesto por un bit wise que indica los destinos a donde esta dirigido el mensaje.

La función debe retornar 1 si existe un mensaje para el destino solicitante y 0 en caso que la lista este vacía o no haya mensajes para el solicitante.

Ejemplo de uso:

```
.....  
data_t dato;  
.....  
ret= popDest ( __otros parámetros__ , &dato , DEST_03);
```

Función: **voidliberar** (__ parámetros__);

La función **liberar** debe vaciar la cola de mensajes, liberar todos los recursos y reestablecer las variables correspondientes indicando que la cola de mensajes está vacía.

Notas generales:

a.-

uint32_t es una definición de tipo
typedef unsigned int uint32_t;

b.- *no es necesario que realice el main(), sólo debe realizar las funciones aquí solicitadas y aquellas que considere necesario o convenientes.*

c.- *En caso de no saber operar con aritmética a nivel de bits, puede considerar que los destinos están definidos por un dígito decimal, como sigue.*

```
ret= pushMulti ( __otros parámetros__ , &dato , 1346);  
  
indicando que los destinos para este dato son el 1, el 3, el 4 y el 6.  
Misma consideración para la función popDest.
```